

module-14-csv-json

Chapter 14: CSV Files and JSON Data | 21 programs in learning order

Source: Automate the Boring Stuff with Python 1e, Al Sweigart (CC-BY-NC-SA). Syllabus Ch14: csv reader/writer, delimiter, header-stripper, json loads/dumps, weather + excel-to-csv

How to use this guide (beginner path)

1) Read Concepts once. 2) Study the Visuals - redraw each on paper. 3) Follow the Program map. 4) Work programs 01..N in order. [shell]: type every plain line into IDLE, compare with # OUT: comments (errors shown are intentional demos). [script]: save, predict output, run with F5. 5) Drill Interview prep. 6) Teach it: explain each visual aloud without looking.

Concepts in detail

CSV = rows of comma values; JSON = nested dicts/lists as text. csv.reader iterates rows (skip header with next(reader)); csv.writer + writerow() emit; delimiter/lineterminator tune the dialect. json.loads() parses text to Python; json.dumps() serializes back. Capstones: strip a CSV column across many files, fetch + read live weather JSON from an API, and convert Excel sheets to CSVs.

Key ideas to carry forward

- for row in reader: row[0], row[1]... - rows are plain lists.
- Open CSVs with newline="" to avoid blank double-spacing on Windows.
- API flow: requests.get(url) -> raise_for_status() -> json.loads(text) -> drill into dicts.

Classic beginner mistakes

- ! Parsing CSV by split(',') and breaking on quoted commas - use the csv module.
- ! Putting single quotes around JSON keys (JSON demands double quotes).

Study order hint: 01-08 csv read/write, 09-12 header stripper, 13-15 json, 16-20 weather API, 21 excel-to-csv.

Interview prep - questions you must answer cold

Say each answer aloud in under 60 seconds, then prove it with one program from this module.

Q1. How do you read CSVs correctly?

A: `csv.reader` over a file opened with `newline=""`; rows are lists; skip header with `next(reader)`. Never `split(',')` - quoted commas break.

Q2. Write CSVs?

A: `csv.writer + writerow(...)`; set `delimiter/lineterminator` for dialects; 'w' with `newline=""`.

Q3. Header-stripper project logic?

A: Loop CSVs, read rows, write all but first row to `*_stripped` - batch file surgery.

Q4. loads vs load, dumps vs dump?

A: `loads/dumps` work with strings; `load/dump` work with file objects. `s` = string.

Q5. Weather API flow?

A: `requests.get(url) -> raise_for_status() -> json.loads(text) -> walk nested dicts/lists for temp/desc`.

Q6. Excel to CSV?

A: `openpyxl` per sheet, iterate rows, `csv.writer` each sheet to its own `.csv` - the converter capstone.

Teach it to others

The one analogy that sticks

CSV is a spreadsheet photographed as plain text (commas are column fences); JSON is Russian dolls (dicts/lists nested) that loads() unpacks into Python and dumps() repacks for travel.

What your students will get wrong

- x CSV parsing is just split(',') (quoted fields break it).
- x JSON allows single quotes/trailing commas (strict double quotes, no trailing).
- x levels nest obviously (print keys at each level while drilling).

5-minute whiteboard plan

1. Read a 3-row CSV and print row[0].
2. Break split(',') on a quoted field, then fix with csv.
3. Drill a weather JSON one level at a time.
4. Exit ticket: loads or load for a string? (loads).

Visualize it - draw these from memory

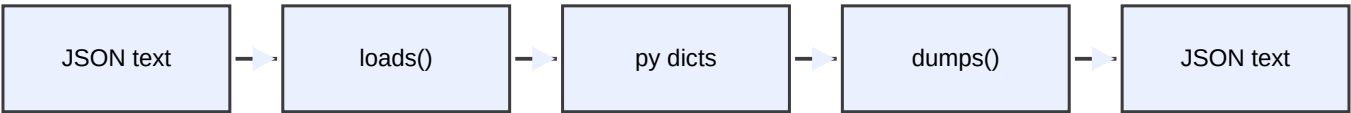
Redraw each figure on paper with labels before you touch code. If you cannot draw it, you cannot teach it.

Visual 1 - CSV grid: rows are lists, fences are commas

name	qty	price
apples	5	2.0
pears	3	1.5

Header row 0, data below. Never split(',') - quoted commas break; csv.reader respects quotes.

Visual 2 - JSON dolls: loads unpacks, dumps repacks



API flow: GET -> raise_for_status -> loads -> drill dicts/lists one level at a time.

More examples - same idea, new clothes

Three bonus drills per module. Cover the output, predict, then run.

X1. CSV column average

```
import csv
with open('data.csv', newline='') as f:
    rows = list(csv.reader(f))
vals = [float(r[1]) for r in rows[1:]]
print(sum(vals) / len(vals))
# EXPECTED: Average of column 2
```

X2. JSON pretty save

```
import json
cfg = {'users': [{'name': 'al', 'admin': True}]}
print(json.dumps(cfg, indent=2))
# EXPECTED: Indented JSON text
```

X3. Count rows per file

```
import csv, glob
for p in glob.glob('*.csv'):
    with open(p, newline='') as f:
        print(p, sum(1 for _ in csv.reader(f)), 'rows')
# EXPECTED: Per-file row counts
```

Program map - all 21 at a glance

```
01 01_csv_module.py
02 02_reader_objects.py
03 03_reader_objects_exempladata.py
04 04_reading_data_from.py
05 05_writer_objects.py
06 06_writer_objects_spam.py
07 07_delimiter_lineterminator_keyword.py
08 08_delimiter_lineterminator_keyword_apples.py
09 09_step_1_loop.py
10 10_step_2_read.py
11 11_step_3_write.py
12 12_step_3_write_removing.py
13 13_json_apis.py
14 14_reading_json_loads.py
15 15_writing_json_dumps.py
16 16_step_1_get.py
17 17_step_2_download.py
18 18_step_3_load.py
19 19_step_3_load_python3.py
20 20_step_3_load_current.py
21 21_excel_csv_converter.py
```

Topic: The CSV Module

01/21 01_csv_module.py [script]

Script: save and run; predict the output first, then compare.

```
4/5/2015 13:34, Apples, 73
4/5/2015 3:41, Cherries, 85
4/6/2015 12:46, Pears, 14
4/8/2015 8:59, Oranges, 52
4/10/2015 2:07, Apples, 152
4/10/2015 18:10, Bananas, 23
4/10/2015 2:40, Strawberries, 98
```

Topic: Reader Objects

02/21 02_reader_objects.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import csv
exampleFile = open('example.csv')
exampleReader = csv.reader(exampleFile)
exampleData = list(exampleReader)
exampleData
# OUT:      [['4/5/2015 13:34', 'Apples', '73'], ['4/5/2015 3:41', 'Cherries', '85'],
# OUT:      ['4/6/2015 12:46', 'Pears', '14'], ['4/8/2015 8:59', 'Oranges', '52'],
# OUT:      ['4/10/2015 2:07', 'Apples', '152'], ['4/10/2015 18:10', 'Bananas', '23'],
# OUT:      ['4/10/2015 2:40', 'Strawberries', '98']]
```

03/21 03_reader_objects_exampdata.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
exampleData[0][0]  
# OUT: '4/5/2015 13:34'  
exampleData[0][1]  
# OUT: 'Apples'  
exampleData[0][2]  
# OUT: '73'  
exampleData[1][1]  
# OUT: 'Cherries'  
exampleData[6][1]  
# OUT: 'Strawberries'
```

Topic: Reading Data from Reader Objects in a for Loop

04/21 04_reading_data_from.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import csv
exampleFile = open('example.csv')
exampleReader = csv.reader(exampleFile)
for row in exampleReader:
# OUT:          print('Row #' + str(exampleReader.line_num) + ' ' + str(row))

# OUT: Row #1 ['4/5/2015 13:34', 'Apples', '73']
# OUT: Row #2 ['4/5/2015 3:41', 'Cherries', '85']
# OUT: Row #3 ['4/6/2015 12:46', 'Pears', '14']
# OUT: Row #4 ['4/8/2015 8:59', 'Oranges', '52']
# OUT: Row #5 ['4/10/2015 2:07', 'Apples', '152']
# OUT: Row #6 ['4/10/2015 18:10', 'Bananas', '23']
# OUT: Row #7 ['4/10/2015 2:40', 'Strawberries', '98']
```

Topic: Writer Objects

05/21 05_writer_objects.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import csv
outputFile = open('output.csv', 'w', newline='')
outputWriter = csv.writer(outputFile)
outputWriter.writerow(['spam', 'eggs', 'bacon', 'ham'])
# OUT:      21
outputWriter.writerow(['Hello, world!', 'eggs', 'bacon', 'ham'])
# OUT:      32
outputWriter.writerow([1, 2, 3.141592, 4])
# OUT:      16
outputFile.close()
```

06/21 06_writer_objects_spam.py [script]

Script: save and run; predict the output first, then compare.

```
spam, eggs, bacon, ham  
"Hello, world!", eggs, bacon, ham  
1, 2, 3.141592, 4
```

Topic: The delimiter and lineterminator Keyword Arguments

07/21 07_delimiter_lineterminator_keyword.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.
Lines starting with >>> are input; other lines are expected output (commented).

```
import csv
csvFile = open('example.tsv', 'w', newline='')
csvWriter = csv.writer(csvFile, delimiter='\t', lineterminator='\n\n')
csvWriter.writerow(['apples', 'oranges', 'grapes'])
# OUT:      24
csvWriter.writerow(['eggs', 'bacon', 'ham'])
# OUT:      17
csvWriter.writerow(['spam', 'spam', 'spam', 'spam', 'spam', 'spam'])
# OUT:      32
csvFile.close()
```

08/21 08_delimiter_lineterminator_keyword_apples.py [script]

Script: save and run; predict the output first, then compare.

apples oranges grapes

eggs bacon ham

spam spam spam spam spam spam

Topic: Step 1: Loop Through Each CSV File

09/21 09_step_1_loop.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# removeCsvHeader.py - Removes the header from all CSV files in the current
# working directory.

import csv, os

os.makedirs('headerRemoved', exist_ok=True)

# Loop through every file in the current working directory.
for csvFilename in os.listdir('.'):
    if not csvFilename.endswith('.csv'):
        continue    # skip non-csv files

    print('Removing header from ' + csvFilename + '...')

    # TODO: Read the CSV file in (skipping first row).

    # TODO: Write out the CSV file.
```

Topic: Step 2: Read in the CSV File

10/21 10_step_2_read.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# removeCsvHeader.py - Removes the header from all CSV files in the current
# working directory.

--snip--
# Read the CSV file in (skipping first row).
csvRows = []
csvFileObj = open(csvFilename)
readerObj = csv.reader(csvFileObj)
for row in readerObj:
    if readerObj.line_num == 1:
        continue    # skip first row
    csvRows.append(row)
csvFileObj.close()

# TODO: Write out the CSV file.
```

Topic: Step 3: Write Out the CSV File Without the First Row

11/21 11_step_3_write.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# removeCsvHeader.py - Removes the header from all CSV files in the current
# working directory.
--snip--

# Loop through every file in the current working directory.
for csvFilename in os.listdir('.'):
    if not csvFilename.endswith('.csv'):
        continue    # skip non-CSV files

--snip--

# Write out the CSV file.
csvFileObj = open(os.path.join('headerRemoved', csvFilename), 'w',
                  newline='')
csvWriter = csv.writer(csvFileObj)
for row in csvRows:
    csvWriter.writerow(row)
csvFileObj.close()
```

12/21 12_step_3_write_removing.py [script]

Script: save and run; predict the output first, then compare.

```
Removing header from NAICS_data_1048.csv...  
Removing header from NAICS_data_1218.csv...  
--snip--  
Removing header from NAICS_data_9834.csv...  
Removing header from NAICS_data_9986.csv...
```

Topic: JSON and APIs

13/21 13_json_apis.py [script]

Script: save and run; predict the output first, then compare.

```
{"name": "Zophie", "isCat": true,  
  "miceCaught": 0, "napsTaken": 37.5,  
  "felineIQ": null}
```

Topic: Reading JSON with the loads() Function

14/21 14_reading_json_loads.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
stringOfJsonData = '{"name": "Zophie", "isCat": true, "miceCaught": 0,  
# OUT: "felineIQ": null}'  
import json  
jsonDataAsPythonValue = json.loads(stringOfJsonData)  
jsonDataAsPythonValue  
# OUT: {'isCat': True, 'miceCaught': 0, 'name': 'Zophie', 'felineIQ': None}
```

Topic: Writing JSON with the dumps() Function

15/21 15_writing_json_dumps.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
pythonValue = {'isCat': True, 'miceCaught': 0, 'name': 'Zophie',
```

```
# OUT: 'felineIQ': None}
```

```
import json
```

```
stringOfJsonData = json.dumps(pythonValue)
```

```
stringOfJsonData
```

```
# OUT: '{"isCat": true, "felineIQ": null, "miceCaught": 0, "name": "Zophie" }'
```

Topic: Step 1: Get Location from the Command Line Argument

16/21 16_step_1_get.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# quickWeather.py - Prints the weather for a location from the command line.

import json, requests, sys

# Compute location from command line arguments.
if len(sys.argv) < 2:
    print('Usage: quickWeather.py location')
    sys.exit()
location = ' '.join(sys.argv[1:])

# TODO: Download the JSON data from OpenWeatherMap.org's API.

# TODO: Load JSON data into a Python variable.
```

Topic: Step 2: Download the JSON Data

17/21 17_step_2_download.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# quickWeather.py - Prints the weather for a location from the command line.

--snip--

# Download the JSON data from OpenWeatherMap.org's API.
url = 'http://api.openweathermap.org/data/2.5/forecast/daily?q=%s&cnt=3' % (location)
response = requests.get(url)
response.raise_for_status()

# TODO: Load JSON data into a Python variable.
```

Topic: Step 3: Load JSON Data and Print Weather

18/21 18_step_3_load.py [script]

Script: save and run; predict the output first, then compare.

```
{'city': {'coord': {'lat': 37.7771, 'lon': -122.42},
          'country': 'United States of America',
          'id': '5391959',
          'name': 'San Francisco',
          'population': 0},
 'cnt': 3,
 'cod': '200',
 'list': [{'clouds': 0,
           'deg': 233,
           'dt': 1402344000,
           'humidity': 58,
           'pressure': 1012.23,
           'speed': 1.96,
           'temp': {'day': 302.29,
                    'eve': 296.46,
                    'max': 302.29,
                    'min': 289.77,
                    'morn': 294.59,
                    'night': 289.77},
           'weather': [{'description': 'sky is clear',
                        'icon': '01d'}]}],
 'sys': {'type': 1,
         'message': 1,
         'offset': 0,
         'sunrise': 1402338000,
         'sunset': 1402356000},
 'timezone': -420,
 'units': 'SI'}
```

19/21 19_step_3_load_python3.py [script]

Script: save and run; predict the output first, then compare.

```
! python3
# quickWeather.py - Prints the weather for a location from the command line.

--snip--

# Load JSON data into a Python variable.
weatherData = json.loads(response.text)
# Print weather descriptions.
w = weatherData['list']
print('Current weather in %s:' % (location))
print(w[0]['weather'][0]['main'], '-', w[0]['weather'][0]['description'])
print()
print('Tomorrow:')
print(w[1]['weather'][0]['main'], '-', w[1]['weather'][0]['description'])
print()
print('Day after tomorrow:')
print(w[2]['weather'][0]['main'], '-', w[2]['weather'][0]['description'])
```

20/21 20_step_3_load_current.py [script]

Script: save and run; predict the output first, then compare.

Current weather in San Francisco, CA:

Clear - sky is clear

Tomorrow:

Clouds - few clouds

Day after tomorrow:

Clear - sky is clear

Topic: Excel-to-CSV Converter

21/21 21_excel_csv_converter.py [script]

Script: save and run; predict the output first, then compare.

```
for excelFile in os.listdir('.'):
    # Skip non-xlsx files, load the workbook object.
    for sheetName in wb.get_sheet_names():
        # Loop through every sheet in the workbook.
        sheet = wb.get_sheet_by_name(sheetName)

        # Create the CSV filename from the Excel filename and sheet title.
        # Create the csv.writer object for this CSV file.

        # Loop through every row in the sheet.
        for rowNum in range(1, sheet.get_highest_row() + 1):
            rowData = []    # append each cell to this list
            # Loop through each cell in the row.
            for colNum in range(1, sheet.get_highest_column() + 1):
                # Append each cell's data to rowData.

            # Write the rowData list to the CSV file.

csvFile.close()
```